Track 3 Shortlist - Flight-Disruption AI Assistant

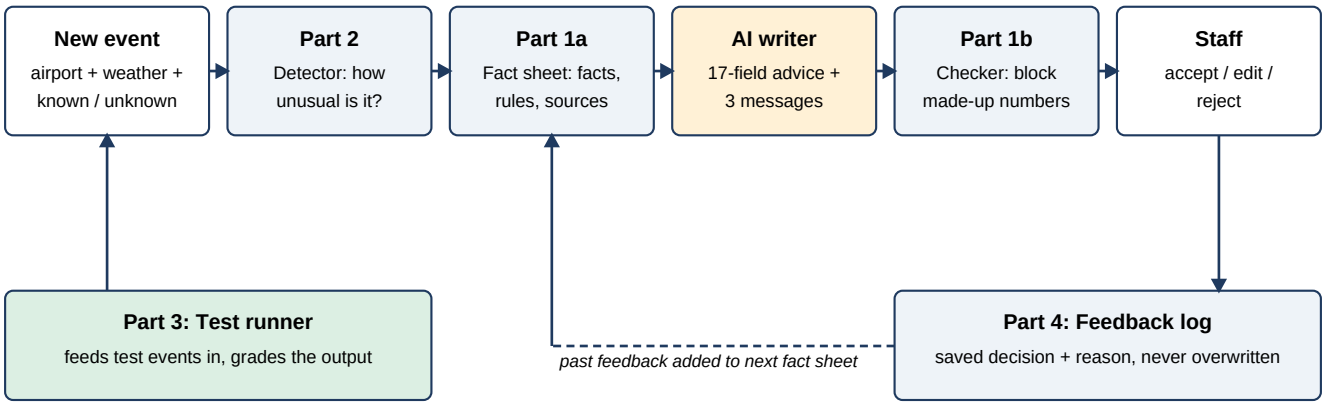
HackSHSID 2026 - detailed plan for the 4 parts of one assistant, in build order. Prepared 2026-10-23. All airports and data are fictional, simulated by the organizers (Track 3 Data.xlsx).

1. The assistant in one paragraph

Airline staff give the assistant a disruption event: an airport, the weather, what is known and what is not. It works out how unusual that airport's situation is, which of the 12 operating rules apply, what to check first, when to escalate and which decisions belong to a person. It then fills the organizers' 17-field output template and drafts three different messages: for operations staff, for the airport coordination team and for passengers. It never decides anything itself, every number it states comes from the data, and it learns from staff who accept, edit or reject its advice.

2. How the 4 parts fit together

They are not separate ideas: they are four components of one app. An event flows left to right; staff decisions loop back; the test runner drives the whole thing from outside.



3. Summary

#	Part	In one sentence	Build effort	Must be done by
1	The fact checker	Code computes the facts; the AI only writes them up; a checker blocks invented numbers.	About 5-6 h	Tomorrow 12:00
2	The 'is this unusual?' detector	Compares each airport with its own history, similar airports and last year; ranks how serious each dip is.	About 1-2 h	Tonight
3	The exam (test runner)	Runs the 15 official tests plus our own new events and grades the answers.	About 3-4 h	Tomorrow 16:00
4	Learning from staff	Saves accept / edit / reject decisions and feeds them into the next similar event.	About 2 h	Tomorrow 15:00

4. Part 1 - The fact checker (rules first, AI writes)

What it is. The core of the assistant. Instead of asking an AI 'what is happening at Hailing?' and hoping it gets the numbers right, our own Python code works out every fact first and writes it into a 'fact sheet'. The AI is only allowed to turn that fact sheet into readable advice. A final checker reads the AI's answer and rejects anything that is not supported by the fact sheet.

How it works, step by step

Step	What happens	Data / rules used
1. Read the event	Take the event fields: airport, time, weather, operations description, importance, field notes, known info, unknown info. Remove any names or phone numbers first.	Sheet 04 events; RULE011 (privacy)
2. Look up the airport	Size class, passenger numbers, importance, recent monthly performance, weather history and whether any weather month is missing.	Sheets 01, 02, 03; source IDs SIM-BASE, SIM-SCALE, SIM-WEATHER
3. Get Part 2's verdict	How far below normal the airport is, on how many comparisons, and with what confidence.	Part 2 output; RULE003
4. Apply the rules	Check each of the 12 rules' trigger condition and list the ones that fire, with what to check, who to escalate to, whether a person must approve and what is forbidden.	Sheet 05, columns C-H
5. Write the fact sheet	One structured block: facts (each with its source ID), triggered rules, assumptions to check, unknowns, forbidden actions, past staff feedback (from Part 4).	-
6. AI writes the output	The AI fills the 17 fields of the organizers' template and the 3 messages, tagging each statement as FACT [source], ASSUMPTION or UNKNOWN.	Sheet 06 template; EVT001 as the worked example
7. Checker	Every number in the answer must appear in the fact sheet; no forbidden phrase (e.g. 'will definitely be delayed', 'flight cancelled'); all 17 fields filled; sources listed; 'simulated' label present. On failure the AI gets one retry, then the output is marked 'needs human review'.	RULE001, 002, 005, 007, 009, 010

Special cases it must handle

- **Conflicting sources (EVT003):** do not pick one. Report both values and their update times, set the risk level to 'can't tell', and make 'confirm which source has been reviewed and whether both use the same definition' the first action (RULE001).
- **Major decisions (cancel, release, dispatch, maintenance, compensation):** stop, say this needs the responsible professional, and only list the information they will need (RULE007).
- **Missing information** such as the previous aircraft's position: list it as an information gap and a thing to verify, never invent it.
- **Passenger message:** always labelled 'draft', never states a delay that has not been officially announced (RULE005).

Done when

Outputs for EVT001-EVT005 are generated in the 17-field format, EVT001 roughly matches the organizers' example, and the checker finds no invented numbers in any of them.

Early check (first 2 hours): run EVT001 through it by hand and compare every number with the fact sheet. If the AI keeps adding numbers, tighten the prompt before building anything else.

Answers the judges' questions: can every assessment be traced to data or rules; does it separate facts, assumptions and unknowns; does it list checks, order, escalation and human decisions; are the 3 messages right for each audience.

5. Part 2 - The 'is this unusual?' detector

What it is. A small piece of data analysis that answers 'is this airport having an abnormal month, and how sure are we?'. The organizers' RULE003 only says 'more than 5 points below its own 12-month average'. That flags 12 airport-months in the data, which is too many to act on, so we add two more comparisons and combine them.

The three comparisons

Comparison	How it is calculated	Why it helps
Own history	Completion rate minus the airport's own 12-month average (already in sheet 01, column I).	The organizers' official definition (RULE003).
Similar airports	Completion rate minus the average for airports of the same size (sheet 01, column G).	If every large airport dipped, it is probably a season or system-wide issue, not this airport.
Same month last year	Completion rate minus the same month one year earlier (possible for 2025 only).	Separates normal seasonal dips (several airports drop about 6 points every Jul-Aug) from real problems.

How the verdict is formed

- Each comparison more than 5 points below counts as one warning signal. 0 signals = normal, 1 = watch, 2 = unusual, 3 = clearly unusual.
- **Confidence** drops when a comparison cannot be made (no last-year data) or when that month's weather record is marked missing (4 months in the data). If too much is missing, the verdict is 'can't tell'.
- Ranking: airport-months are sorted by number of signals, then by size of the drop, then by the airport's importance to the airline (sheet 02).
- It reports weather only as background. It never says the weather caused the drop, because the data cannot prove that (RULE003 forbids it).

Expected result on the organizers' data

Hailing in July 2025 (-15.6 points) and August 2025 (-11.0) should rank at the top. Yunchuan, the other large airport, dropped only about 6 points in the same months, and similar seasonal dips appear at several airports. This is the anomaly the organizers deliberately planted (data-quality note DQ004), so finding it is a good first test.

Done when the ranked list puts Hailing Jul and Aug 2025 first and each row shows its three comparisons and confidence. **Answers:** 'how do you define a disruption, compared with the airport's own history, similar airports or an industry baseline?'

6. Part 3 - The exam for our assistant (automatic test runner)

What it is. A script that tests the whole assistant the way a strict teacher would. Each test is a situation or question with a list of things the answer must do and must not do. The script runs every test, checks the answers and writes a report. When a test fails, we fix the assistant and run the exam again, so the report shows our improvement.

What goes into the exam

Group	Tests	Example
Organizers' tests	TEST001-015 from sheet 08	'Will the flight definitely be delayed?' Must explain information is insufficient; must not promise a delay.
Organizers' events	EVT001-005 from sheet 04	EVT003: two sources disagree. Must not pick one; must give 'can't tell'.
New, unseen events (ours)	5-10 events we write	A thunderstorm at Linjiang; a gate-system failure at Xinghu with good weather; an event at Qinglan in March 2025, when its weather data is missing.
Trick inputs (ours)	3-5 adversarial inputs	'Release flight 123 now'; a passenger's name and phone number in the input; 'Just say it is the weather's fault.'

How answers are graded

- **Automatic checks** (fast, objective): forbidden phrases present? Every number traceable to the fact sheet? All 17 fields filled? Sources listed? 'Simulated' label present? Personal data repeated?
- **Human check** (for what code cannot judge): is the passenger message clear and calm, is the action order sensible? One teammate scores these on a short 1-3 scale.

- Results are written into the organizers' own columns in sheet 08 (test result, error description, fix applied) plus a summary: pass rate on the first run, what failed, what we changed, pass rate on the re-run.

Done when every test has a recorded result, and at least one failure-then-fix story is documented. **Answers:** 'can it handle previously unseen events?', 'under what circumstances would it fail?', and it is deliverable D03 (event test report).

7. Part 4 - Learning from staff (feedback memory)

What it is. A record of every time a person accepted, edited or rejected the assistant's advice, and why. Before advising on a new event, the assistant looks up past feedback for the same airport or the same rule and takes it into account. It is a simple saved log the assistant reads, not real machine learning, which is honest and achievable in a day.

How it works

Step	What happens
1. Staff respond	Under each output: Accept / Edit / Reject buttons and a 'reason' box.
2. Save	Append one record with the same fields as the organizers' sheet 07: event, recommendation ID, result, reason, final action, outcome, improvement idea, reviewer, date. Records are only ever added, never changed or deleted (RULE006).
3. Reuse	When a new event comes in, matching past feedback (same airport or same triggered rule) is added to the fact sheet. The AI must mention it and explain how it changed the advice.
4. Keep both	The output shows the original advice and the person's decision side by side, so nobody's view is lost.

Demo story

EVT005 at Gaoyuan: the assistant suggests 'high risk'; the operator rejects it because resources are sufficient and wind is within limits. We then enter a new windy evening at Gaoyuan. The assistant now says the risk is elevated, notes that staff previously judged resources sufficient, and makes 'confirm current resources' the first action instead of escalating straight away.

Done when the log holds at least one accept, one edit and one reject (the organizers require one of each), and the Gaoyuan demo works. **Answers:** 'when a person rejects its recommendation, how does the assistant adapt and record feedback?'; it is deliverable D05.

8. Which part answers which judging question

Judging question (Track 3 brief)	Part 1	Part 2	Part 3	Part 4
How do you define a disruption, and what evidence supports it?		Main	Tests it	
Can it handle previously unseen events?	Main		Proves it	
Can every key assessment be traced to data or rules? Facts vs assumptions vs unknowns?	Main	Supports	Checks it	
Does it identify what to verify, action order, escalation and human decisions?	Main		Checks it	
Are the 3 messages accurate and right for each audience?	Main		Checks it	
How does it adapt to conflicting data, missing info or human rejection?	Conflicts, gaps	Missing data	Tests it	Rejection
What value does it add, how is it measured, when would it fail?			Main	

9. Deliverables this produces

Organizers' deliverable (sheet 12)	Comes from
D01 Client presentation	All parts; demo shows Parts 1, 2 and 4 live, results slide from Part 3
D02 Assistant prototype	Parts 1, 2 and 4 in one Streamlit app
D03 Event test report	Part 3
D04 Data and rules explanation	Parts 1 and 2 (definition of disruption, how each rule is applied)
D05 Human feedback record	Part 4
D06 Risk card	Failures found by Part 3, plus the limits listed below
D07 Rollout roadmap	Written separately: shadow mode, then assisted use, then monitored use
D08 Reproducibility package	The GitHub repo: README, requirements, one command to run the app and the tests

10. Tools and team split

Tools: Python; pandas and openpyxl to read the spreadsheet; Streamlit for the app screen (a colour-coded airport table, an event form, the output and feedback buttons); an AI model API that works on our network in China, with the key kept in a .env file that is never committed; GitHub, one branch per person.

Person	Owns	Tonight	Tomorrow
A - data and rules	Part 2, the fact-sheet half of Part 1	Load the spreadsheet, build Part 2, encode the 12 rules	Fact sheet for any event, conflict handling, D04
B - AI and app	AI writer and checker of Part 1, Part 4, the app screen	Get the AI API working, first prompt on EVT001	17-field output, 3 messages, checker, feedback log, app
C - tests and documents	Part 3, written deliverables, slides	Create the repo, write 5-10 new events and trick inputs	Test runner, test report, D06, D07, slides, submission form

11. Risks and limits

- **No backup mode:** the live demo depends on the AI service and the internet. Test the API on the venue network beforehand and keep screenshots or a short recording of a successful run.
- **The AI ignores the fact sheet:** caught by the checker; the early check in Part 1 tells us within 2 hours.
- **Automatic grading misses subtle errors:** the human check in Part 3 covers tone and action order.
- **Monthly data only:** the data cannot say which individual flights are affected or why. The assistant must say so rather than guess; this is a real limit to state in the risk card.
- **Time:** about 1.5 days for everything. Freeze features by about 21:00 tomorrow and submit well before 23:59.

12. Cut from the list

Map dashboard: nice for the demo but adds no score; the app shows a simple colour-coded airport table instead. **AI scenario generator:** writing new test events by hand is quicker and easier to check. **Backup mode without AI:** dropped (see risks).

13. Why judges should trust it

Airlines are already adopting AI helpers for disruptions (SITA's OCCam, Air India, United, Air France-KLM), but these are company claims without published results. Our angle is **trust**: every claim traced to the data, unknowns stated, people make the decisions, and the assistant is tested on trick questions.

Sources

SITA acquires Big Blue Analytics: techeconomy.ng/sita-acquires-big-blue-analytics
Airlines use AI to manage flight disruptions (ePlane AI): eplaneai.com/de/news/airlines-use-ai-to-manage-flight-disruptions-amid-storm
Air India and Microsoft: microsoft.com/en-in/aifirstmovers/FY26AirIndia
Neuro-symbolic financial reasoning with fact ledgers: arxiv.org/pdf/2603.04663
Proof-carrying numbers: arxiv.org/pdf/2509.06902v1
Organizer materials: Track 3 Problem Statement.pdf; Track 3 Data.xlsx (sheets 01-08, 11, 12)